

EDS Lab Assignments

Practical 1

Name: Sudipt kale

PRN : 202501040097

Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m * v$ where:
m is the mass of the object (in kilograms). v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kg m /s) (rounded up to 2 decimal places).

Code:

```
mass = float(input()) velocity =  
float(input()) momentum =  
mass * velocity  
print(f"{momentum :.2f} kg m /s")
```

Output:

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

- m is the mass of the object (in kilograms).
- v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

Calculate...

```

1 m=float(input())
2 v=float(input())
3 p=m*v
4 print(f"{p:.2f}kgm/s")

```

Average time: 0.002 s
Maximum time: 0.003 s

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 3 ms Debug

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s	50.00kgm/s

Test case 2 2 ms

2. Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.
- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```

import math
n=int(input()) sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else: print("Invalid")

```

Output:

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

- $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

```

1 n = int(input())
2 if(n<=9):
3     print(n**2)
4 elif(n>=10 and n<=99):
5     print(f"{n**(1/2):.2f}")
6 elif(n>=100 and n<=999):
7     print(f"{n**(1/3):.2f}")
8 else:
9     print("Invalid")

```

Average time: 0.001 s, 0.85 ms | Maximum time: 0.002 s, 2.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case	Expected output	Actual output
Test case 1	9	9
Test case 2	81	81
Test case 3		

3. Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format `YYYY-MM-DD`.
- The second line contains the second date in the format `YYYY-MM-DD`.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```

from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2 - d1
print(difference.days)

```

Output:

1.13. Days Between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format `YYYY-MM-DD`.
- The second line contains the second date in the format `YYYY-MM-DD`.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases +

```
1 # from datetime import date
2
3 #write your code here...
4 from datetime import date
5 import datetime
6
7 date1_str = input()
8 date2_str = input()
9 y1,m1,d1 = map(int,date1_str.split("-"))
10 y2,m2,d2 = map(int,date2_str.split("-"))
11 date1 = datetime.date(y1,m1,d1)
12 date2 = datetime.date(y2,m2,d2)
13
14 difference = date2-date1
15 print(difference.days)
```

Average time: 0.001 s, 1.25 ms | Maximum time: 0.002 s, 2.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1	Debug	Test
Expected output		Actual output
2014-07-02		2014-07-02
2014-11-02		2014-11-02
123		123

Test case 2 1 ms

4. Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer. □

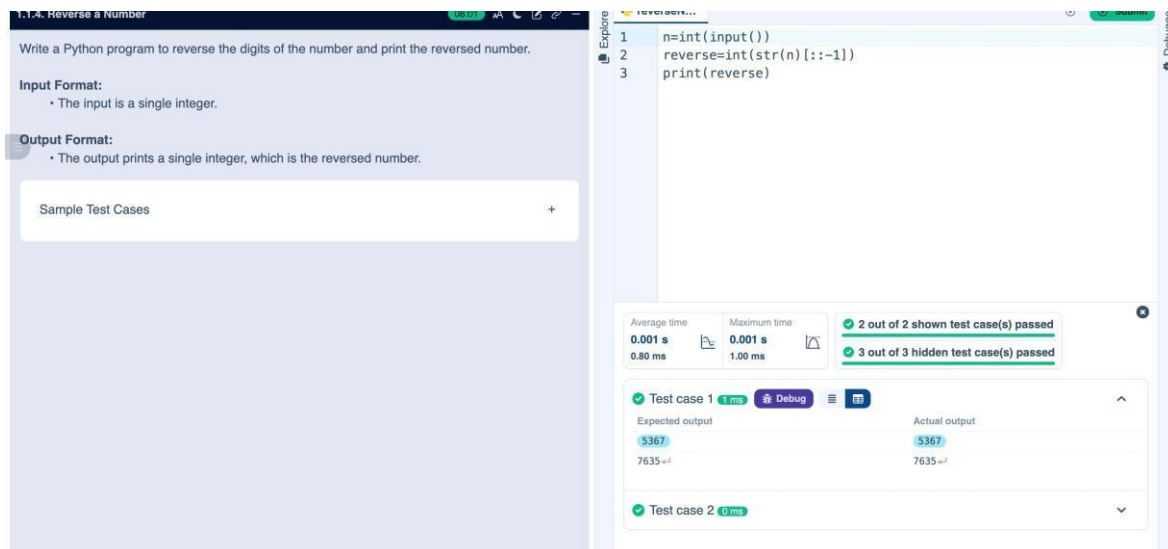
Output Format:

- The output prints a single integer, which is the reversed number. □

Code:

```
n=int(input()) reverse=int(str(n)[::-1])
print(reverse)
```

Output:



5. Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format: `<number>`
`x <multiplier> = <result>`

Code:

```
n = int(input())
```

```
for i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays a Python IDE interface. On the left, a panel titled "1.1.5. Multiplication Table" contains the problem description: "Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10." It specifies the input format (an integer), the output format (a table of products), and a note about using "x" for multiplication. Below this is a section for "Sample Test Cases".

The main editor shows the following Python code:

```
1 n = int(input())
2 for i in range(1,11):
3     print(n,"x",i,"=", n*i)
```

On the right, a "Debug" panel shows the execution results. It indicates that the program passed all test cases: "2 out of 2 shown test case(s) passed" and "2 out of 2 hidden test case(s) passed". The execution time is 0.002 s. Below this, a table compares the "Expected output" with the "Actual output" for Test case 1 (input 8).

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

1.2.1 Pass or Fail

Problem Statement:

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

```
□ "Fail".
```

Code :

```
def calculate(marks, total_courses):
    if any(mark < 40 for mark in marks):
        return "Fail"

    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75: grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)

num_courses = int(input())
marks = list(map(int, input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:", f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output :

The screenshot displays a code editor with the following Python code and its execution results:

```
1 n=int(input())
2 mark=list(map(int,input().split()))
3 fail=False
4 for i in mark:
5     if i<40:
6         fail=True
7         break
8 if(fail==True):
9     print("Fail")
10 else:
11     agg=sum(mark)/n
12     if(agg>75):
13         print("Aggregate Percentage:",f"{agg:.2f}")
14         print("Grade: Distinction")
15 elif(agg>=60):
```

The execution output shows the following test cases passed:

- 5 out of 5 shown test case(s) passed
- 5 out of 5 hidden test case(s) passed

The test case details are as follows:

Test case 1	Expected output	Actual output
4	4	4
55 65 78 89	55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division	Grade: First Division

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

```
Code: def fibonacci(n,
a=0, b=1):
    if n==0: return
        a
    else:
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())
for i in
range(n):
    print(fibonacci(i),end=" ")
```

Output:

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

```

1  # def fibonacci(n):
2  #     if(n<2):
3  #         return n
4  #     else:
5  #         return fibonacci(n-1) + fibonacci(n-2)
6
7
8  # n = int(input())
9  # for i in range(1, n+1):
10 #     print(fibonacci(i), end=" ")
11
12 def fibonacci(n):
13     if(n<2):
14         return n
15     else:

```

Average time: 0.005 s
 Maximum time: 0.016 s
 5.00 ms 16.00 ms

2 out of 2 shown test case(s) passed
 2 out of 2 hidden test case(s) passed

Test case 1 (1 ms) Debug

Expected output	Actual output
5	5
0 1 1 2 3	0 1 1 2 3

Test case 2 (1 ms)

1.2.3 Pattern -1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```

rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*", end=" ")
    print()

```

Output:

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

```

1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print("*",end=" ")
5     print("")
6

```

Average time
0.001 s
1.17 ms

Maximum time
0.002 s
2.00 ms

2 out of 2 shown test case(s) passed
4 out of 4 hidden test case(s) passed

Test case 1

Expected output

Actual output

```

5
*
* *
* * *
* * * *
* * * * *

```

1.2.4 Pattern -2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```

n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()

```

Output:

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

```
1 n = int(input())
2 for i in range(1,n+1):
3     a=1
4     for j in range(1,i+1):
5         print(a,end=" ")
6         a=a+1
7     print("")
```